Cursor Prompt: HKUST(GZ) Floating Building PV & Load Data Processing

Project Context

I am doing an MPhil research on energy dispatch strategy evaluation for a floating zero-carbon building (HKUST(GZ) Waterfront Living Lab). I need to generate two time-series datasets for the full year 2023 (8760 hours):

1. Hourly PV generation (kW) — from PVGIS CSV files
2. Hourly building load (kW) — from ASHRAE 90.1 schedule coefficients

Then I need to run three dispatch strategies (RBC, MILP, MPC) on these datasets and compare their performance.

Task 1: Process PVGIS Data → PV Generation Curve (pv_generation.py)

Input Files (all in same folder, PVGIS ERA5, year 2023)

- pvgis_south_45.csv — Slope 45°, Azimuth 0° (South-facing)
- pvgis_east_45.csv — Slope 45°, Azimuth -90° (East-facing)
- pvgis_west_45.csv — Slope 45°, Azimuth 90° (West-facing)
- pvgis_north_45.csv — Slope 45°, Azimuth 180° (North-facing)
- pvgis_deck.csv — Slope 0°, Azimuth 0° (Horizontal)

PVGIS CSV Format

- Skip first 8 rows (metadata)
- Columns: time, Gb(i), Gd(i), Gr(i), H_sun, T2m, WS10m, Int
- time format: YYYYMMDD:HHMM (e.g. 20230101:0030)
- Gb(i): beam irradiance W/m²
- Gd(i): diffuse irradiance W/m²
- Gr(i): reflected irradiance W/m²
- T2m: ambient temperature °C
- Total irradiance G(i) = Gb(i) + Gd(i) + Gr(i)

Building BIPV Parameters

Sphere surface (total 290 m², split equally across 4 orientations = 72.5 m² each):

- Opaque HJT panels: 250 m² total, efficiency = 0.21

- Transparent CdTe panels: 40 m² total, efficiency = 0.12
- Sphere opaque fraction per orientation: 250/290 = 0.862
- Sphere transparent fraction per orientation: 40/290 = 0.138

Horizontal surfaces (use pvgis_deck.csv):

- Outer floating deck: area = 67.6 m², efficiency = 0.21 (HJT)
- Glass balustrade: area = 62.4 m², efficiency = 0.12 (CdTe)

PV Calculation Formula

For each hour:
```
G_total = Gb(i) + Gd(i) + Gr(i)        # W/m²
temp_factor = 1 + (-0.004) * (T2m - 25)   # temperature correction
power = G_total * area * efficiency * temp_factor / 1000  # kW
power = max(0, power)                  # no negative power
```

Water Surface Microclimate Correction (apply to all zones)

```
albedo_factor = 1.05  # water surface reflection adds ~5% to irradiance
cooling_factor = 1.012  # 3°C cooling × 0.4%/°C efficiency gain = +1.2%
corrected_power = power * albedo_factor * cooling_factor
```

(These factors represent the mid-range of literature values:
albedo 0.06-0.15, temp reduction 2-5°C)

Output

- Calculate total hourly PV output = sum of all zones
- Save to pv_generation_8760.csv with columns: datetime, pv_kW
- Print annual total and compare with design estimate of 55,580 kWh
- Plot daily and monthly PV generation for visual verification

---

Task 2: Generate Building Load Curve (load_generation.py)

Known Parameters (from design institute)

- With HVAC: total power = 47.7 kW, annual runtime = 896 hours
- Without HVAC: total power = 26.1 kW, annual runtime = 679 hours
- HVAC power = 47.7 - 26.1 = 21.6 kW
- Base load (lighting + sockets + pumps) = 26.1 kW
- Annual total consumption = 60,461 kWh

Seasonal Assignment (Nansha climate)

- HVAC ON months: May, June, July, August, September (months 5-9) → use 47.7 kW as base
- HVAC OFF months: October to April (months 10-4) → use 26.1 kW as base

ASHRAE 90.1 Schedule Coefficients

Apply these hourly multipliers to the base power.
Different coefficients for weekdays vs weekends.

Weekday schedule (index = hour 0-23): hour_coefficients_weekday = [0.30, # 00:00 0.30, # 01:00 0.30, # 02:00 0.30, # 03:00 0.30, # 04:00 0.30, # 05:00 0.30, # 06:00 0.50, # 07:00 0.70, # 08:00 1.00, # 09:00 1.00, # 10:00 1.00, # 11:00 0.80, # 12:00 (lunch dip) 1.00, # 13:00 1.00, # 14:00 1.00, # 15:00 1.00, # 16:00 0.80, # 17:00 0.60, # 18:00 0.50, # 19:00 0.40, # 20:00 0.35, # 21:00 0.30, # 22:00 0.30, # 23:00]

Weekend/holiday schedule: hour_coefficients_weekend = [0.20, # 00:00 0.20, # 01:00 0.20, # 02:00 0.20, # 03:00 0.20, # 04:00 0.20, # 05:00 0.20, # 06:00 0.25, # 07:00 0.30, # 08:00 0.40, # 09:00 0.40, # 10:00 0.40, # 11:00 0.35, # 12:00 0.40, # 13:00 0.40, # 14:00 0.40, # 15:00 0.35, # 16:00 0.30, # 17:00 0.25, # 18:00 0.20, # 19:00 0.20, # 20:00 0.20, # 21:00 0.20, # 22:00 0.20, # 23:00]

Load Calculation

For each hour in 2023:

1. Determine if HVAC season (month 5-9) → base_power = 47.7 else 26.1
2. Determine if weekday or weekend → select coefficient array
3. load_kW = base_power * hour_coefficient[hour_of_day]

Output

- Save to load_8760.csv with columns: datetime, load_kW
- Print annual total and compare with design estimate of 60,461 kWh
- Plot daily and monthly load for visual verification

Task 3: Dispatch Strategy Comparison (dispatch.py)

System Parameters

- BESS capacity: 215 kWh
- BESS max charge/discharge rate: 50 kW (assume)
- BESS initial SOC: 0.5 (50%)
- BESS min SOC: 0.1 (10%), max SOC: 0.9 (90%)
- BESS round-trip efficiency: 0.92

- Grid: always available (except islanding scenario)
- Electricity price: 0.8 CNY/kWh (import), 0.4 CNY/kWh (export)

Input

- pv_generation_8760.csv
- load_8760.csv
- Both aligned to same datetime index (8760 rows)

Strategy 1: Rule-Based Control (RBC)

Logic for each hour:
net = pv_kW - load_kW
if net > 0: # surplus
charge battery up to max SOC and max charge rate
export remaining surplus to grid
else: # deficit
discharge battery down to min SOC and max discharge rate
import remaining deficit from grid

Strategy 2: MILP Optimal Dispatch

Use Python library: pip install pulp

- Decision variables per hour: charge(t), discharge(t), grid_import(t), grid_export(t)
- Objective: minimize total annual grid import cost - export revenue
- Constraints: SOC dynamics, capacity limits, charge/discharge limits, power balance
- Assume perfect foresight (knows full 8760-hour profile in advance)
- Solve using CBC solver (included with PuLP)
- Note: for 8760 hours this may be slow, consider solving month by month and concatenating results

Strategy 3: Model Predictive Control (MPC)

- Rolling window: 24-hour prediction horizon
- At each hour t: solve MILP for next 24 hours, execute only first decision
- Move to t+1, repeat
- Use actual PV and load data as "perfect short-term forecast" (this represents best-case MPC performance)
- This creates 8760 individual 24-hour MILP problems

Performance Metrics (calculate for each strategy)

1. Annual grid purchase cost (CNY) = sum(grid_import * 0.8)
2. Annual grid export revenue (CNY) = sum(grid_export * 0.4)
3. Net annual cost (CNY) = purchase cost - export revenue
4. PV self-consumption rate (%) = (pv_used_directly + pv_via_battery) / total_pv * 100
5. Battery cycle count = total_charge_kWh / 215
6. Self-sufficiency rate (%) = (load - grid_import) / load * 100

Islanding Scenario (separate analysis)

- Define islanding load = 13 kW (essential services only, ~50% of base load)
- Start condition: grid disconnects at hour 0 of a typical day
- For each strategy: how many hours until battery depletes below min SOC?
- Test on: best day (high PV), worst day (low PV), average day (medium PV)
- Select these days from pv_generation_8760.csv by daily total

Output

- results_comparison.csv: all metrics for all three strategies side by side
- plots: (a) One week example showing SOC and power flows for each strategy (b) Monthly cost comparison bar chart (c) Islanding autonomy duration bar chart

---

Task 4: Sensitivity Analysis for Water Surface Microclimate (sensitivity.py)

Parameters to vary

- albedo_factor: [1.00, 1.03, 1.05, 1.08, 1.10] (represents 0% to 10% irradiance enhancement from water reflection)
- cooling_factor: [1.000, 1.004, 1.008, 1.012, 1.016, 1.020] (represents 0°C to 5°C module cooling at 0.4%/°C)

Analysis

For each combination of (albedo_factor, cooling_factor):

1. Recalculate PV generation (scale the base pv_kW by albedo_factor * cooling_factor)
2. Run RBC strategy (fastest) to get performance metrics
3. Record: annual_pv_total, self_consumption_rate, net_cost

Output

- sensitivity_results.csv: full grid of results
- heatmap plot: net_cost vs (albedo_factor, cooling_factor)
- Report: range of PV output variation and its impact on dispatch economics

---

File Structure Expected

```
project/
├── data/
│   ├── pvgis_south_45.csv
│   ├── pvgis_east_45.csv
│   ├── pvgis_west_45.csv
│   ├── pvgis_north_45.csv
│   └── pvgis_deck.csv
├── pv_generation.py
├── load_generation.py
├── dispatch.py
├── sensitivity.py
├── output/
│   ├── pv_generation_8760.csv
│   ├── load_8760.csv
│   ├── results_comparison.csv
│   └── sensitivity_results.csv
└── plots/
```

Required Libraries

pip install pandas numpy matplotlib pulp scipy

Important Notes for Cursor

1. Always use pandas datetime for time alignment between PV and load data
2. PVGIS CSV has metadata rows at top — skip rows until you find the header row containing 'time' as first column
3. For MILP with 8760 hours, solve by month (12 x ~730 hours) to avoid memory issues, then concatenate
4. MPC with 8760 rolling windows will be slow — add a progress bar using tqdm
5. All power values in kW, all energy values in kWh
6. SOC tracking: SOC(t+1) = SOC(t) + charge(t)*eff*/215 - *discharge(t)*/(*eff*215)
7. Print validation checkpoints at end of each script showing key totals